



Advanced cargo usage

Cargo commands and scripts

First, what is **build.rs**? Does Cargo Run It Automatically?

If you place a file named **build.rs** in the root of your project (at the same level as **Cargo.toml**), **Cargo will automatically run it before compiling your crate.**

Yes, Cargo runs **build.rs**, and yes, this is a **convention**.

How Does Cargo Know to Run It?

Cargo follows **naming and layout conventions** to understand your project structure:

File	Purpose
src/main.rs	Main binary
src/lib.rs	Main library
tests/*.rs	Integration tests
examples/*.rs	Example binaries
benches/*.rs	Benchmarks (see note below)
build.rs	Custom build script

Cargo automatically looks for a file called **build.rs** in the project root. If it exists, Cargo:

- 1. Compiles **build.rs** into a standalone executable.
- 2. Runs that executable **before compiling your main crate**.
- 3. Uses its output (like environment variables, generated code, or linker flags) during the build.

This is called a **custom build script**, and it gives you fine-grained control over the build process.

What Can You Do in `build.rs`?

The `build.rs` script allows you to:

- Generate Rust source code at build time.
- Link to native libraries (C/C++/Fortran).
- Detect system configuration (OS, CPU features, etc.).
- Set compile-time environment variables.
- Control when Cargo should re-run the build.
- Print warnings or errors during compilation.

Example: Correct Use of `OUT_DIR`

Generated files must be written to the directory specified by the `OUT_DIR` environment variable — **not into `src/`**.

Here's the correct way:

`build.rs`

```
// build.rs
// build.rs
use std::env;
use std::fs;
use std::path::Path;
use chrono::Utc;

fn main() {
    // Get the output directory (set by Cargo)
    let out_dir = env::var("OUT_DIR").expect("OUT_DIR not set by Cargo");
    let dest_path = Path::new(&out_dir).join("greeting.rs");

    let now = Utc::now();
    let name = "Alice";

    // Generate Rust code
    let greeting = format!(
        r#"
        pub fn greet() -> &'static str {{
            "Hello, {}! This project was built on {}."
        }}
        "#,
        name,
        now.format("%Y-%m-%d %H:%M:%S UTC")
    );

    // Write to OUT_DIR
    fs::write(&dest_path, greeting)
        .expect("Failed to write generated file");

    // Tell Cargo when to re-run the build
```

```
println!("cargo:rerun-if-changed=build.rs");  
}
```

In `src/main.rs`

```
// src/main.rs  
include!(concat!(env!("OUT_DIR"), "/greeting.rs"));  
  
fn main() {  
    println!("{}", greet());  
}
```

This keeps generated artifacts in the `target/` directory, **outside your source tree**, avoiding version control noise and ensuring clean builds.

Custom Cargo Commands and Scripts

Cargo, Rust's powerful build system and package manager, isn't just for building and testing. One of its most underappreciated features is its extensibility: you can create **custom Cargo commands** using simple naming conventions and scripts.

This allows teams and developers to automate common tasks—like formatting, linting, releasing, or running dev servers—with a clean, consistent interface: `cargo <your-command>`.

How Custom Cargo Commands Work

Cargo looks for executables in your `PATH` that follow the naming pattern:

```
cargo-<subcommand>
```

When you run:

```
cargo mytool
```

Cargo searches for an executable named `cargo-mytool` in your `PATH`. If found, it runs it as if you had typed `cargo-mytool`.

This means you can write custom tools in **Rust, Bash, Python, or any language**—as long as the executable is discoverable and named correctly.

Example 1: A Simple Bash Script as `cargo-greet`

Let's create a custom command `cargo greet` that prints a friendly message.

1. **Create the script** (e.g., `cargo-greet` without extension):

```
#!/bin/bash
echo "Hello from your Rust project!"
```

2. Make it executable:

```
chmod +x cargo-greet
```

3. Move it to a directory in your `PATH`, like `~/.cargo/bin/` (recommended):

```
mv cargo-greet ~/.cargo/bin/
```

`~/.cargo/bin` is automatically added to your `PATH` when you install Rust via `rustup`.

4. Now run it:

```
cargo greet
# Output: Hello from your Rust project!
```

Success! You've created a custom Cargo command.

Example 2: A Rust Binary as `cargo-checklist`

Let's write a more advanced tool in Rust that checks common project tasks.

1. Create a new binary project:

```
cargo new cargo-checklist --bin
cd cargo-checklist
```

2. Replace `src/main.rs`:

```
use std::process::Command;

fn main() {
    println!("Running project checklist...\n");

    // Check formatting
    let fmt = Command::new("cargo").args(["fmt", "--check"]).output().unwrap();
```

```

    if fmt.status.success() {
        println!(" Code is formatted");
    } else {
        println!(" Code needs formatting");
    }

    // Run clippy
    let clippy = Command::new("cargo").args(["clippy", "--all-targets", "--", "-D", "warnings"]).output().unwrap();
    if clippy.status.success() {
        println!(" Clippy passed");
    } else {
        println!(" Clippy found issues");
    }

    // Run tests
    let test = Command::new("cargo").arg("test").output().unwrap();
    if test.status.success() {
        println!(" All tests passed");
    } else {
        println!(" Tests failed");
    }

    println!("\n Checklist complete!");
}

```

And edit `Cargo.toml`:

```

[package]
name = "cargo-checklist"
version = "0.1.0"
edition = "2021"
description = "A custom cargo command to check code quality: fmt, clippy, and tests"
repository = "https://github.com/yourusername/cargo-checklist"

[dependencies]

```

Note: Your package name **MUST** be `cargo-checklist`! If you use a different `package name` then you will need an array-formed section named `[[bin]]` with name and path attributes.

Array-formed sections: In TOML, `[[bin]]` (with double square brackets) denotes an **array of tables** rather than a single table. This allows you to declare multiple binaries in the same `Cargo.toml`, each as a separate element of the `bin` array. If you used `[bin]` (single square brackets), you would only get a single table called `bin`, and subsequent declarations would overwrite the previous one. So, whenever you want to list more than one executable (or even just one, but following the Cargo standard), you should use `[[bin]]`. Cargo will iterate over each item in this array and generate a binary for each one.

3. Build and install:

```
cargo install --path .
```

This compiles and installs `cargo-checklist` into `~/.cargo/bin`.

4. Use it in any Rust project:

```
cargo checklist
```

Output:

```
Running project checklist...

OK - Code is formatted
Ok - Clippy passed
Ok - All tests passed

Checklist complete!
```

Using Scripts in Your Project (Local-Only Commands)

Sometimes you don't want to install globally. You can define project-specific scripts using a `scripts/` directory and a wrapper.

Example: `scripts/cargo-dev-server`

```
#!/bin/bash
echo "Starting dev server..."
cargo watch -x "run"
```

Make it executable and run via:

```
./scripts/cargo-dev-server
```

Or add `scripts/` to `PATH` temporarily:

```
export PATH=./scripts:$PATH
cargo dev-server
```

Tip: Use `just` (<https://github.com/casey/just>) or `cargo-make` for more robust task running.

Tools That Use This Pattern

Many popular tools are implemented this way:

- `cargo fmt` → `rustfmt` (via `cargo-fmt`)
- `cargo clippy` → `clippy` (via `cargo-clippy`)
- `cargo watch` → external binary
- `cargo sqlx` → `cargo-sqlx`

You can even see what custom commands are available:

```
cargo --list
```

Output:

```
Available commands:
  build      Compile a local package and all of its dependencies
  check      Check a local package and all of its dependencies for
errors
  clean      Remove artifacts that cargo has generated in the past
  ...
  greet      (from ~/.cargo/bin/cargo-greet)
  checklist  (from ~/.cargo/bin/cargo-checklist)
```

Best Practices

- Name your binaries `cargo-<command>` exactly.
- Place them in `~/.cargo/bin` for global availability.
- Use `cargo install --path .` for local tools.
- Handle errors gracefully and provide helpful output.
- Consider cross-platform compatibility (avoid Bash-only scripts if distributing).